PRAKTIKUM SISTEM OPERASI

MODUL 6

Proses Sekuensial dan Konkuren

Oleh:

Candra Dinata - 2311104061

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

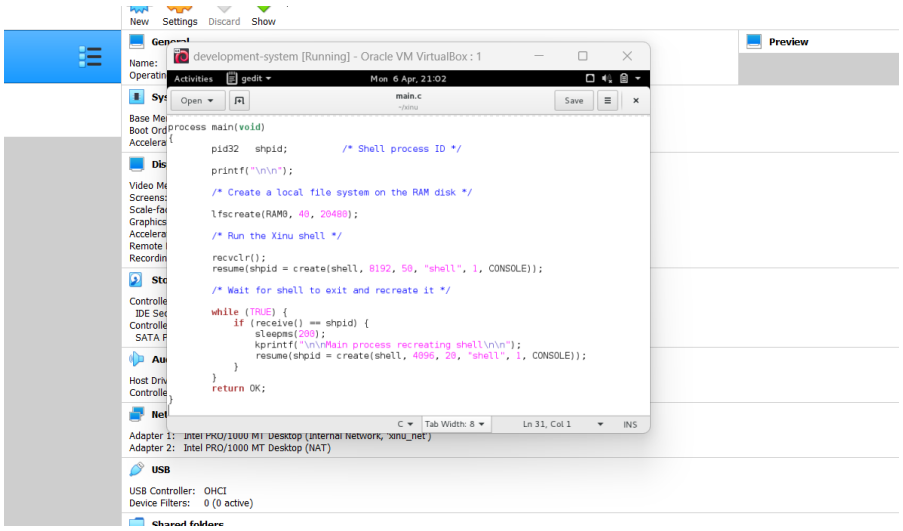
JURNAL/TP

1. Di **Xinu**, jumlah maksimal proses **dibatasi secara software** melalui sebuah konstanta, biasanya bernama:
   `#define NPROC 50`
   Lokasi file:
   `/include/conf.h`

   Secara default:
   `#define NPROC 50`
   Artinya:
   Xinu hanya bisa membuat **maksimal 50 proses** dalam satu waktu.

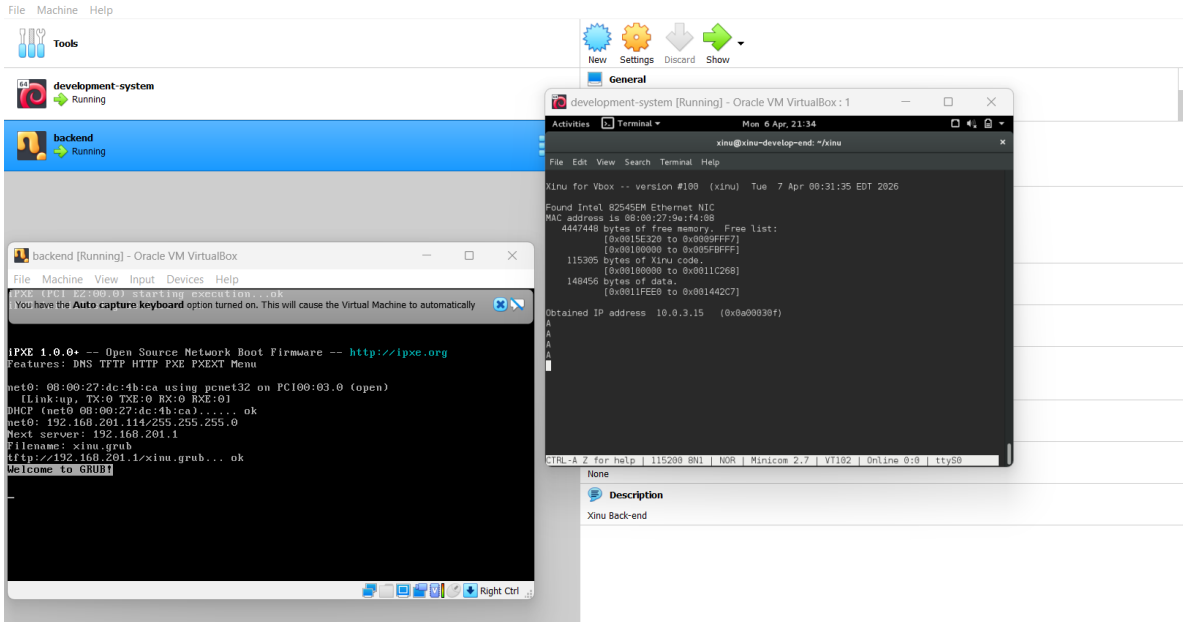
   Mengubah Menjadi 150 Proses
   Buka file:
   `vi /xinu/include/conf.h`
   2. Cari baris:
   `#define NPROC 50`
   3. Ubah menjadi:
   `#define NPROC 150`

   Setelah Diubah
   Jangan lupa **recompile Xinu**:
   ```
   make clean
   make
   ```

2. 

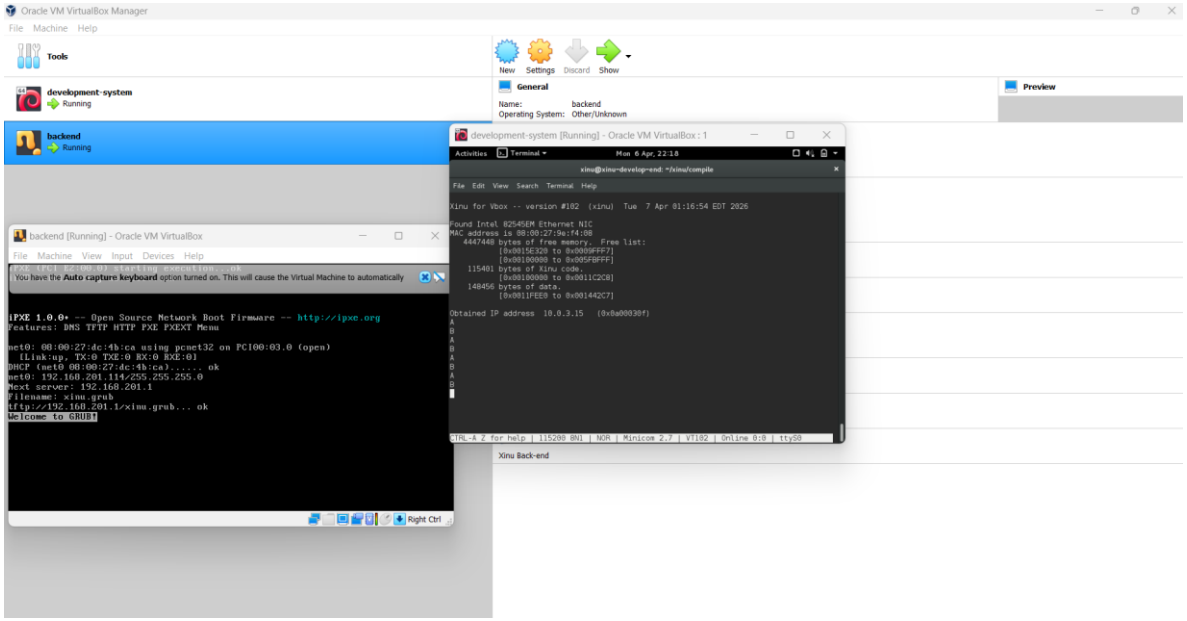
   Back up main.c

Lalu Jalankan kode main_ex1!



Kode program Xinu di atas merupakan contoh sederhana yang menunjukkan eksekusi fungsi secara berurutan tanpa menggunakan mekanisme *multitasking* atau pembuatan proses baru. Pada fungsi main(), terdapat pemanggilan dua fungsi yaitu sndA() dan sndB(). Namun, karena sndA() dipanggil terlebih dahulu dan berisi perulangan tak hingga (while(1)), maka program akan terus mengeksekusi isi dari sndA() tanpa pernah kembali ke main() untuk menjalankan sndB(). Fungsi sndA() sendiri mencetak karakter "A" ke layar setiap 1 detik menggunakan printf() dan sleepms(1000). Sementara itu, fungsi sndB() memiliki struktur yang sama, yaitu mencetak "B" setiap 1 detik, tetapi fungsi ini tidak pernah dijalankan karena eksekusi program sudah terjebak di dalam loop tak hingga pada sndA(). Dengan demikian, output yang dihasilkan hanyalah huruf "A" yang muncul terus-menerus setiap detik. Program ini secara tidak langsung menunjukkan bahwa tanpa pembuatan proses terpisah (misalnya menggunakan create() dan resume() di Xinu), pemanggilan fungsi biasa tidak akan berjalan secara paralel.

3. Lalu Jalankan kode main_ex2!

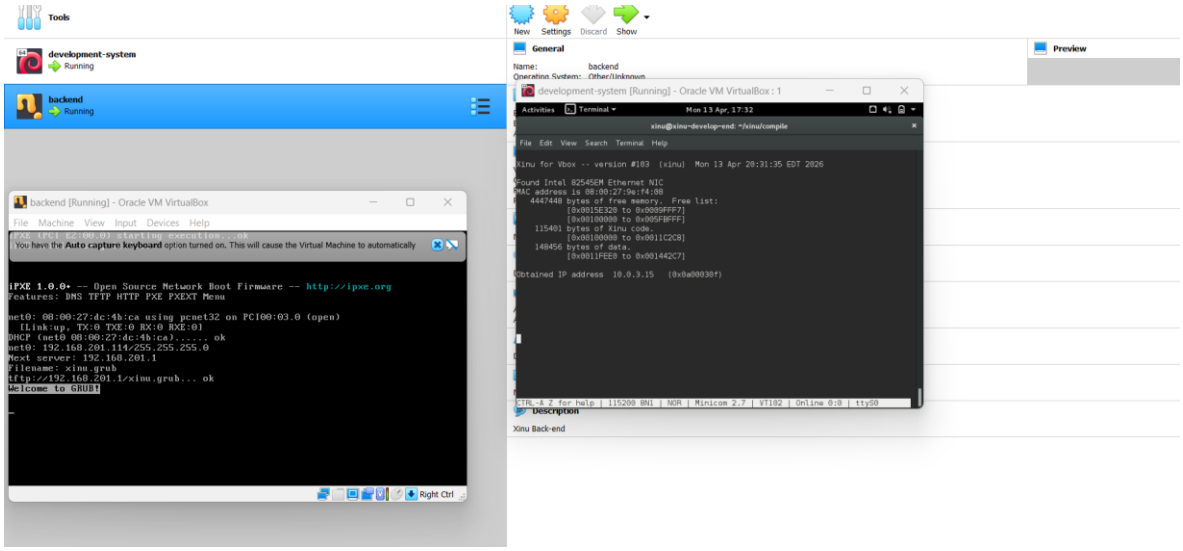


Kode program Xinu di atas merupakan contoh implementasi *multitasking* dengan membuat dua proses yang berjalan secara bersamaan (konkuren). Pada fungsi main(), digunakan fungsi create() untuk membuat dua proses baru, yaitu sndA dan sndB. Parameter pada create() meliputi nama fungsi yang dijalankan, ukuran stack (1024), prioritas proses (20), nama proses (string), dan jumlah argumen (0). Setelah proses dibuat, fungsi resume() digunakan untuk mengaktifkan (menjalankan) proses tersebut sehingga masuk ke dalam *ready queue* dan dapat dijadwalkan oleh scheduler Xinu.

Fungsi sndA() dan sndB() masing-masing berisi perulangan tak hingga (while(1)) yang mencetak karakter "A" dan "B" ke layar setiap 1 detik menggunakan printf() dan sleepms(1000). Karena kedua fungsi dijalankan sebagai proses yang berbeda, maka Xinu akan melakukan *context switching* di antara keduanya. Hal ini memungkinkan output "A" dan "B" muncul secara bergantian di layar, tergantung pada penjadwalan CPU oleh sistem operasi.

Berbeda dengan kode sebelumnya yang hanya menjalankan satu fungsi secara terus-menerus, pada kode ini kedua fungsi benar-benar berjalan secara paralel (konkuren), sehingga menunjukkan konsep dasar manajemen proses pada sistem operasi Xinu.

4. Lalu Jalankan kode main_ex3!

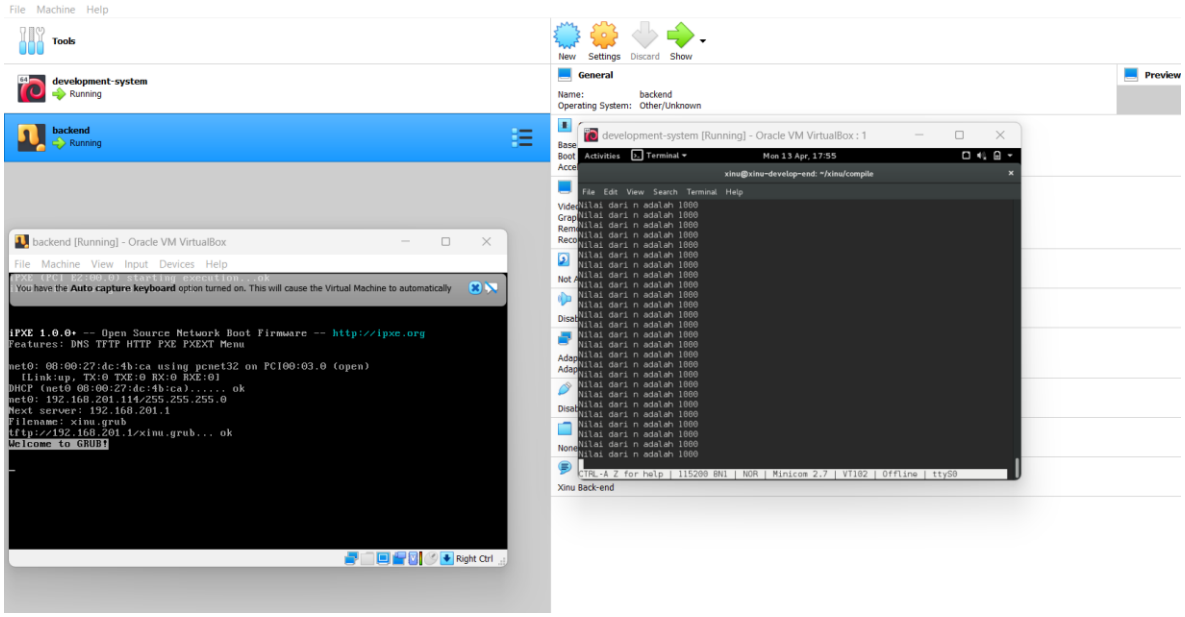


Kode program Xinu di atas menunjukkan penggunaan *multitasking* dengan parameter pada fungsi proses. Pada fungsi main(), dibuat dua proses menggunakan create() yang masing-masing menjalankan fungsi yang sama, yaitu sndChar, tetapi dengan argumen yang berbeda. Proses pertama diberi argumen karakter 'A' dengan nama “Send A”, sedangkan proses kedua diberi argumen 'B' dengan nama “Send B”. Parameter terakhir pada create() yaitu 1 menunjukkan bahwa fungsi sndChar menerima satu argumen, yaitu karakter yang akan dicetak. Setelah dibuat, kedua proses diaktifkan menggunakan resume() sehingga dapat dijalankan oleh scheduler Xinu.

Fungsi sndChar(char c) menerima parameter karakter c, lalu menjalankan perulangan tak hingga (while(1)) yang mencetak karakter tersebut ke layar setiap 1 detik menggunakan printf() dan sleepms(1000). Karena kedua proses berjalan secara konkuren, maka output yang dihasilkan akan berupa karakter "A" dan "B" yang muncul secara bergantian di layar, tergantung pada penjadwalan CPU.

Program ini menunjukkan bahwa dalam Xinu, satu fungsi proses dapat digunakan kembali (*code reuse*) dengan memberikan parameter berbeda, sehingga lebih efisien dibandingkan membuat dua fungsi terpisah. Selain itu, contoh ini juga memperlihatkan bagaimana proses-proses dapat berjalan secara paralel dan saling berbagi CPU melalui mekanisme penjadwalan.

5. Buatlah 2 proses produser dan konsumer. Produser memproduksi angka integer dari 1-1000.



Kode program Xinu di atas menggambarkan konsep dasar interaksi antara *producer* dan *consumer*, namun masih dijalankan secara sekuensial (tidak paralel). Variabel global n diinisialisasi dengan nilai 0 dan digunakan sebagai data bersama. Fungsi produser() berperan sebagai penghasil data, yaitu dengan melakukan increment (n++) sebanyak 1000 kali melalui perulangan for, sehingga setelah fungsi ini selesai dijalankan, nilai n akan menjadi 1000. Selanjutnya, fungsi konsumer() dijalankan, yang bertugas mencetak nilai n sebanyak 1000 kali menggunakan printf().

Karena pada fungsi main() pemanggilan dilakukan secara berurutan (produser() terlebih dahulu, lalu konsumer()), maka tidak terjadi *race condition* atau konflik akses data. Hal ini disebabkan seluruh proses penambahan nilai n sudah selesai sebelum dilakukan pembacaan oleh konsumer(). Akibatnya, output yang ditampilkan akan selalu menunjukkan nilai n yang sama, yaitu 1000, sebanyak 1000 kali.

Program ini menunjukkan bahwa tanpa penggunaan mekanisme proses paralel (seperti create() dan resume()), konsep producer-consumer belum benar-benar terlihat, karena tidak ada eksekusi bersamaan maupun sinkronisasi antar proses.